



به نام خدا

مصطفی چرکزی

آزمایش هشتم پردازش سیگنال های دیجیتال

آزمایش ۸: پردازش تصاویر دیجیتال

هر تصویر دیجیتال در متلب، آرایه‌ای دو بعدی و یا سه بعدی (برای تصاویر رنگی) از اعداد است. در این آزمایش با خواندن تصاویر دیجیتال در متلب، عملیات‌های ابتدایی پردازش بر روی تصاویر، ترکیب و ذخیره تصاویر آشنا خواهیم شد.

دستورات متلب مورد نیاز:

image processing toolbox:

imread	imwrite	imfinfo	imnoise	Mat2gray	Rgb2gray	Im2uint8
im2uint16	imhist					

جعبه ابزار پردازش تصویر :

image processing toolbox:

imread	imwrite	imfinfo	imnoise	Mat2gray	Rgb2gray	Im2uint8
im2uint16	imhist	imfilter	histeq	Filter2	Medfilt2	fspecial

جعبه ابزار پردازش تصویر :

بخش ۱: خواندن تصویر و مشاهده هیستوگرام

فعالیت ۱: الف) با استفاده از دستور imread، تصویر موجود در فولدر آزمایش به نام image1.tif را فراخوانده و در متغیر a ذخیره کنید. ابعاد آرایه تصویر چند است و نوع داده‌های آن چیست؟ ب) تصویر a را با استفاده از دستور imshow مشاهده و سپس هیستوگرام آن را با استفاده از دستور imhist رسم کنید.

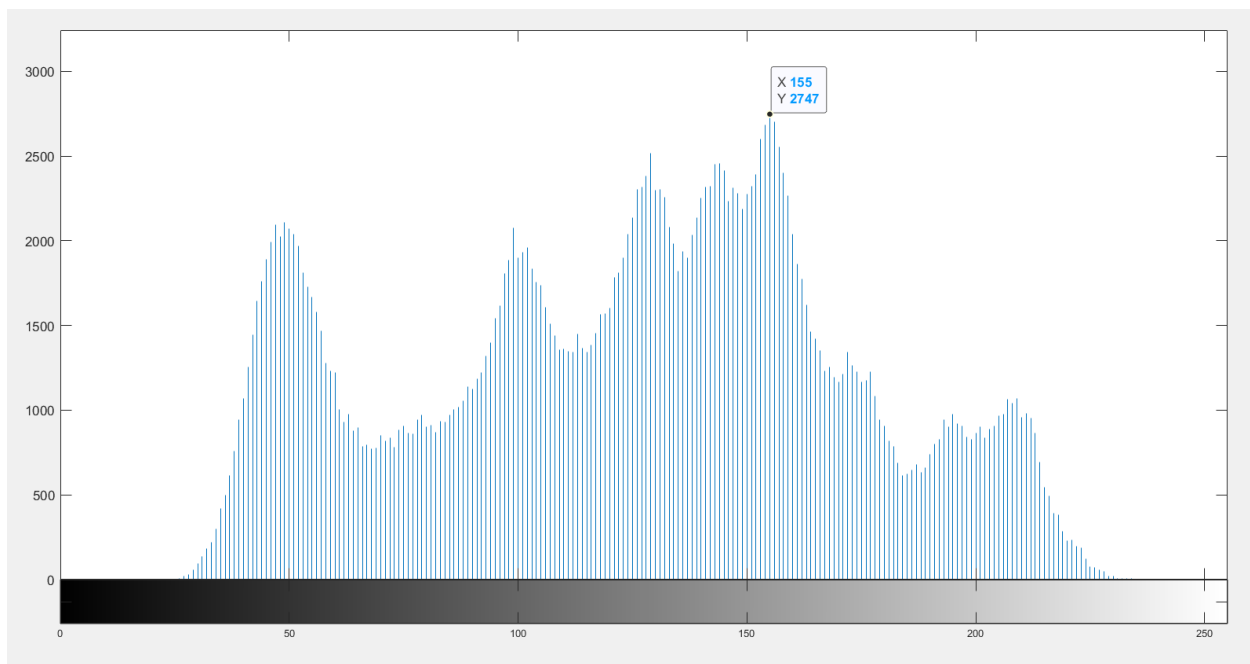
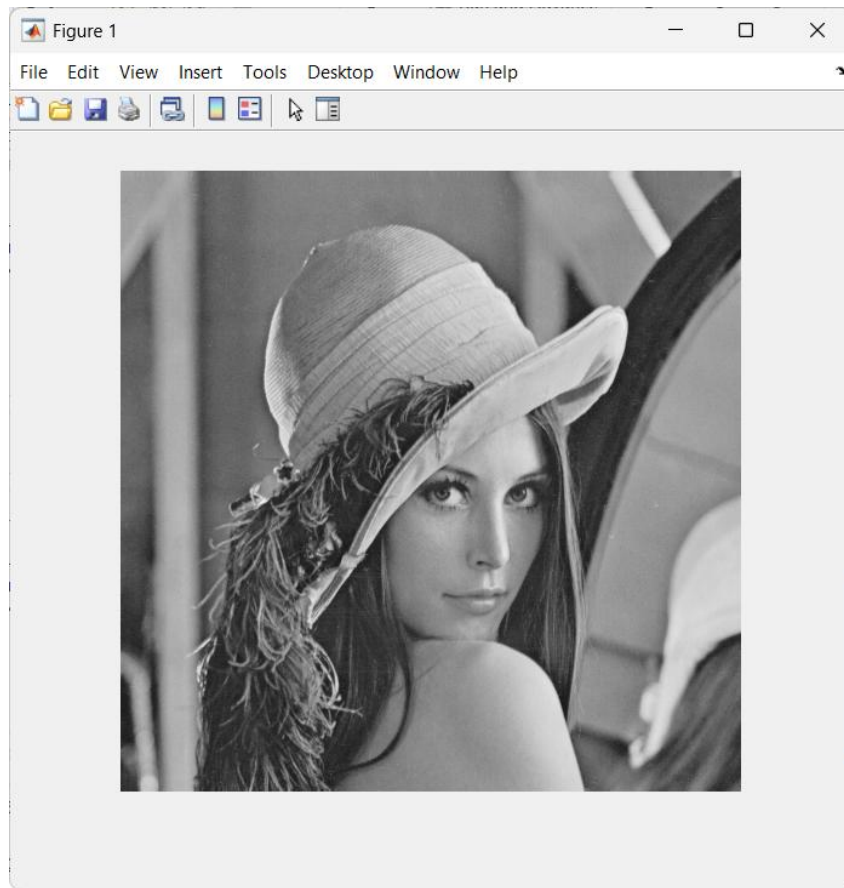
```
clc; clear; close all;  
image_url = "files/image1.tif";  
a = imread(image_url);
```

```
whos a
```

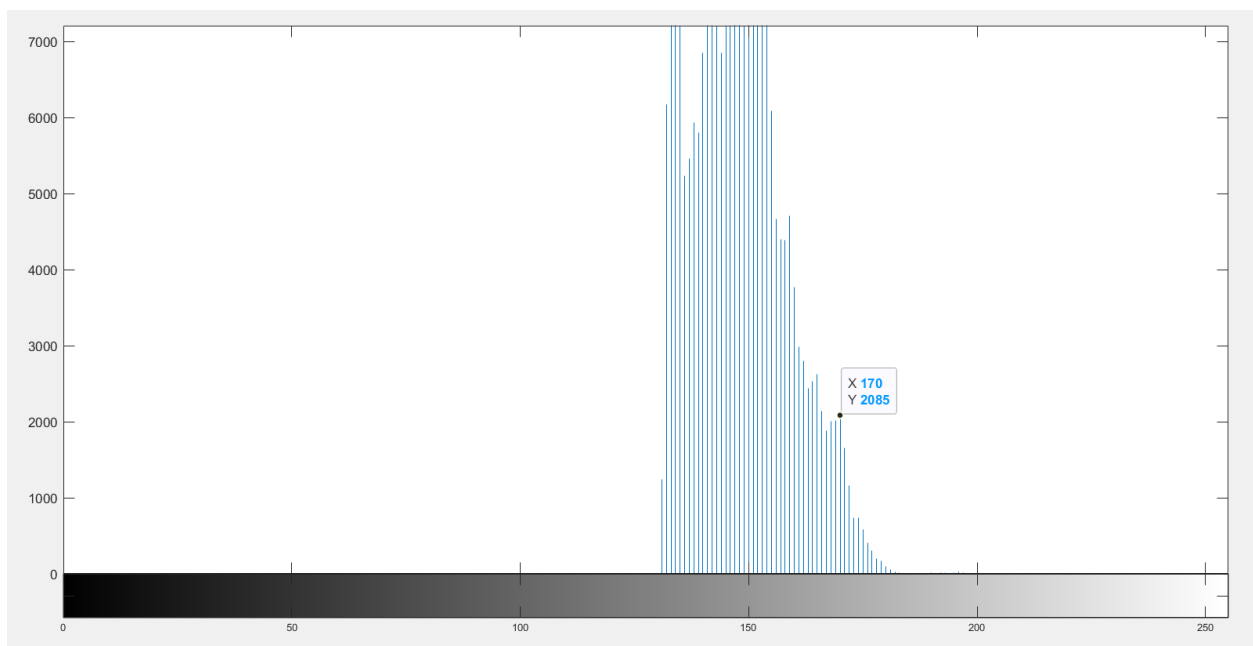
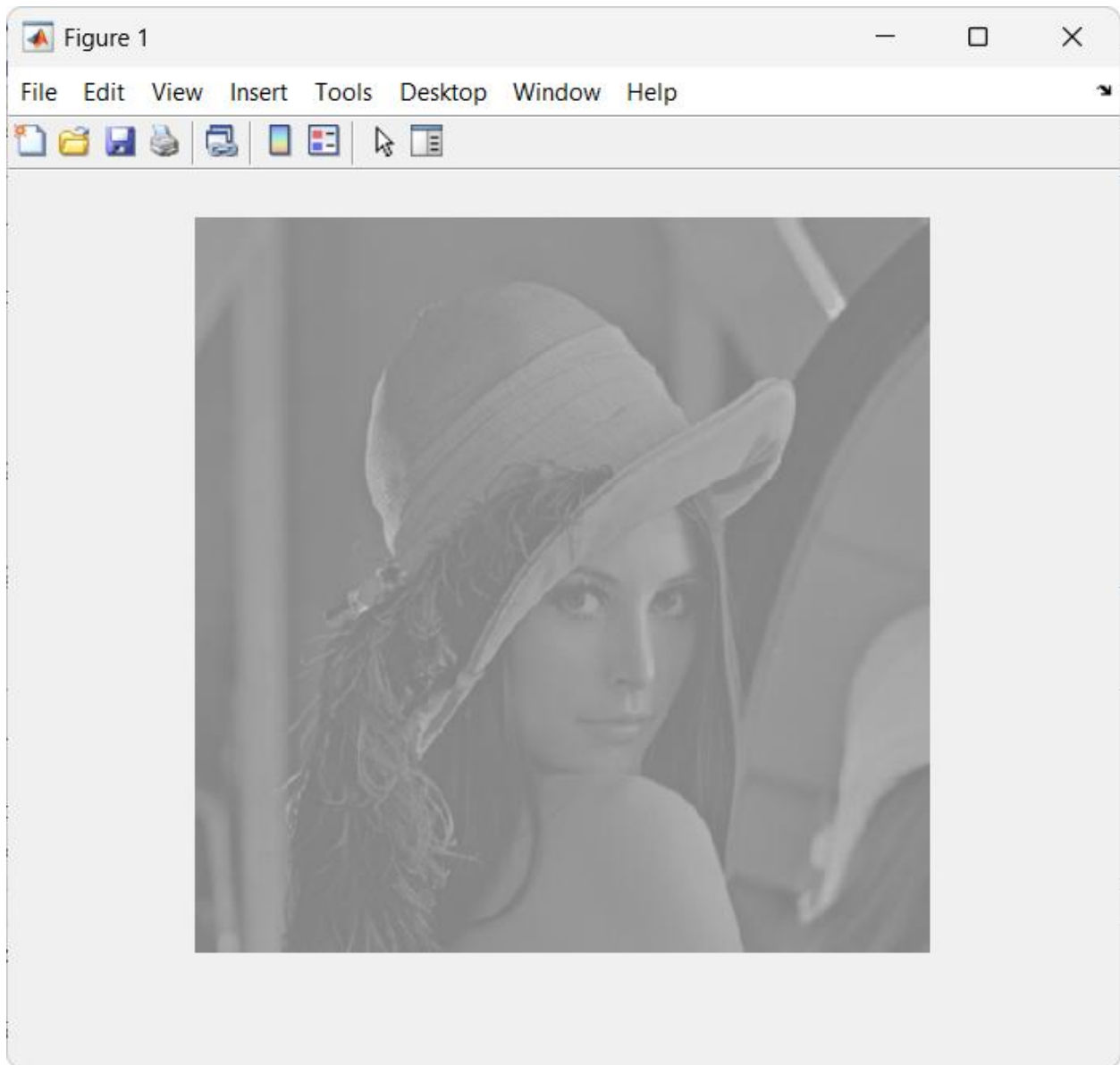
```
figure;  
imshow(a);
```

```
figure;  
imhist(a);
```

Name	Size	Bytes	Class	Attributes
a	512x512	262144	uint8	



فعالیت ۲: فعالیت ۱ را برای تصویر image2 نیز انجام دهید. هیستوگرام این تصویر چه تفاوتی با هیستوگرام تصویر قبل دارد؟ با توجه به تعریف هیستوگرام، این تفاوت را چگونه توجیه میکنید؟



فعالیت ۳: الف) آیا میتوانید تصویر image2 را واضح تر کنید؟ (سعی کنید با استفاده از تابع معادله خط هیستوگرام را پخش کنید). ب) حال دستور histeq را بر روی تصویر اعمال کرده و خروجی و هیستوگرام آن را ترسیم کنید.

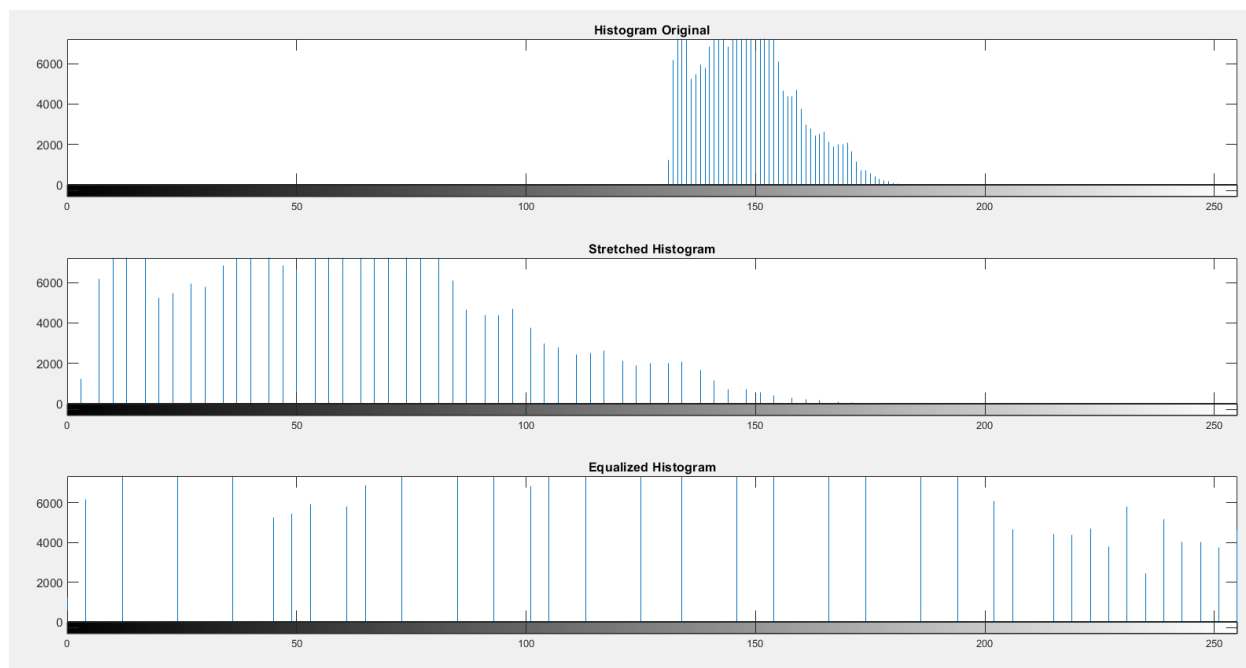
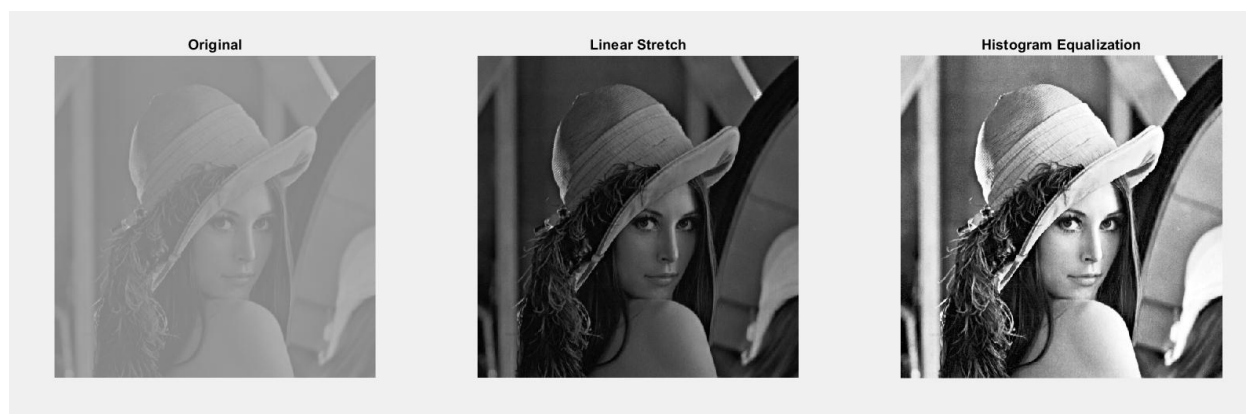
اگر معادله خطی را به صورت زیر فرض کنیم

$$y = ax + b$$

$$x = x_{min} \rightarrow y = 0$$

$$x = x_{max} \rightarrow y = 255$$

$$\begin{cases} 0 = a x_{min} + b \\ 255 = a x_{max} + b \end{cases} \rightarrow a = \frac{255}{x_{max} - x_{min}}, b = -a x_{min}$$



```

image_url = "files/image2.tif";
img2 = imread(image_url);
img2_double = double(img2);

x_min = min(img2_double(:));
x_max = max(img2_double(:));

% Method a: calculating image_stretch using y=ax+b
a = 255 / (x_max - x_min);
b = - a * x_min;
img2_stretch = a * img2_double + b;
img2_stretch = uint8(img2_stretch);

% Method b: Histogram equalization
img2_histeq = histeq(img2);

% Display results
figure('Name', 'Activity 3 - Enhancement');
subplot(1,3,1); imshow(img2); title('Original');
subplot(1,3,2); imshow(img2_stretch); title('Linear Stretch');
subplot(1,3,3); imshow(img2_histeq); title('Histogram Equalization');

figure('Name', 'Activity 3 - Histograms after enhancement');
subplot(3,1,1); imhist(img2); title('Histogram Original');
subplot(3,1,2); imhist(img2_stretch); title('Stretched Histogram');
subplot(3,1,3); imhist(img2_histeq); title('Equalized Histogram');

```

فعالیت ۴: تصویر image3 را خوانده و در متغیر a ذخیره کنید. تصویر را مشاهده کنید. پیکسل‌های مربوط به کانال‌های R و G و B را بصورت جداگانه در سه متغیر ذخیره کرده و نمایش دهید. هیستوگرام مربوط به هر رنگ را نیز نمایش دهید.

```

image_url = "files/image3.png";

a = imread(image_url);
r = a(:,:,1);
g = a(:,:,2);
b = a(:,:,3);

figure;
subplot(2,1,1); imshow(a); title("Main picture");
subplot(2,1,2); imhist(a); title("Main picture Hist");

figure;
subplot(2,1,1); imshow(r); title("R picture");
subplot(2,1,2); imhist(r); title("R picture Hist");

figure;
subplot(2,1,1); imshow(g); title("G picture");
subplot(2,1,2); imhist(g); title("G picture");

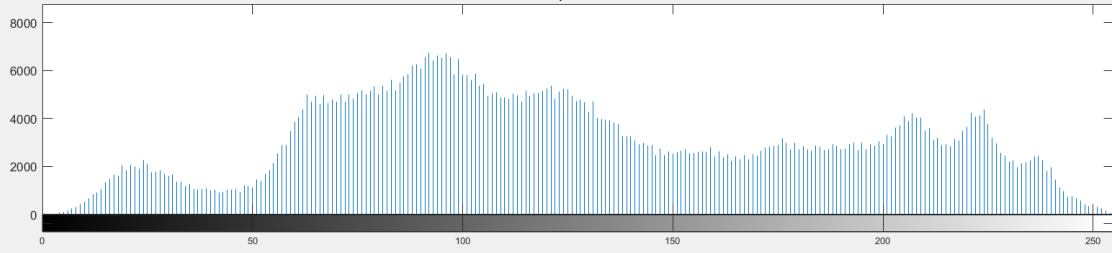
figure;
subplot(2,1,1); imshow(b); title("B picture");
subplot(2,1,2); imhist(b); title("B picture");

```

Main picture



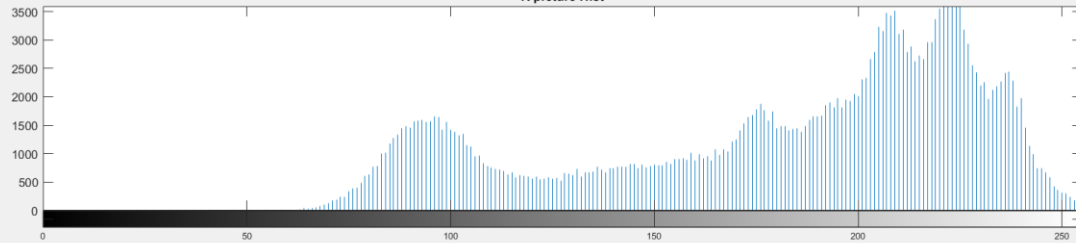
Main picture Hist



R picture



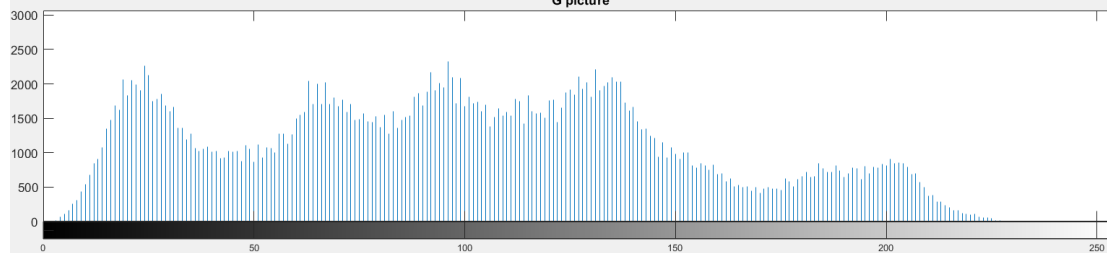
R picture Hist

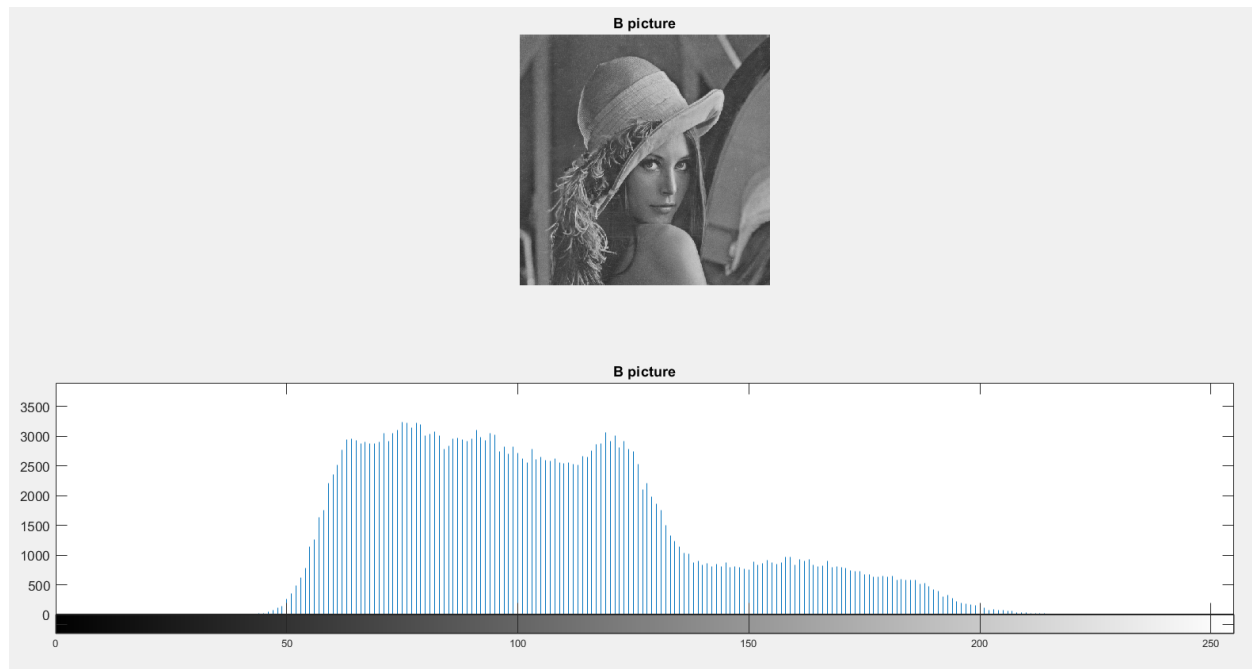


G picture



G picture





بخش ۲: اعمال تبدیل روی تصاویر

فعالیت ۵: تصویر image1.tif را فراخوانی کرده و ابعاد آن را به مقدار 100*100 تغییر دهید و تصویر جدید را مشاهده کنید. هیستوگرام تصویر جدید را با تصویر قبل مقایسه کنید. آیا تفاوتی مشاهده می‌شود؟ چرا؟

```
image_url = "files/image1.tif";  
  
img1 = imread(image_url);  
  
% Resize to 100x100 using default bicubic interpolation  
img_resized = imresize(img1, [100, 100], 'bilinear');  
% 'nearest', 'bilinear', 'bicubic'  
  
% Display results  
figure('Name', 'Activity 5 - Image Resizing');  
subplot(1,2,1);  
imshow(img1);  
title(['Original: ', num2str(size(img1,1)), 'x', num2str(size(img1,2))]);  
  
subplot(1,2,2);  
imshow(img_resized);  
title(['Resized: ', num2str(size(img_resized,1)), 'x',  
num2str(size(img_resized,2))]);
```

کاهش ابعاد از 512x512 به 100x100 باعث از دست رفتن جزئیات ریز می‌شود، اما برای بسیاری از کاربردهای پردازش تصویر مناسب است.

از سایر متد ها مانند nearest, bilinear, bicubic میتوانیم برای resize کردن تصویر استفاده کنیم. طبق داکيومنت تابع resize در متلب این متد ها پشتیبانی میشوند.

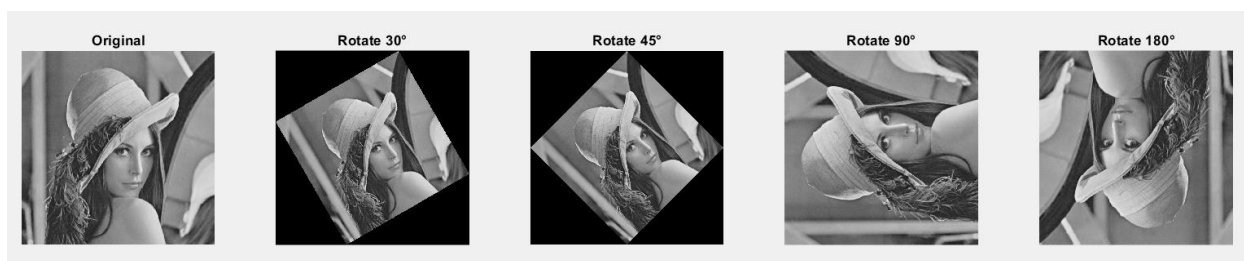
Method	Description
"nearest"	Nearest-neighbor interpolation; the output pixel is assigned the value of the pixel that the point falls within. No other pixels are considered.
"bilinear"	Bilinear interpolation; the output pixel value is a weighted average of pixels in the nearest 2-by-2 neighborhood
"bicubic"	Bicubic interpolation; the output pixel value is a weighted average of pixels in the nearest 4-by-4 neighborhood.
Note Bicubic interpolation can produce pixel values outside the original range.	
Interpolation Kernel	Description
"box"	Box-shaped kernel
"triangle"	Triangular kernel (equivalent to "bilinear")
"cubic"	Cubic kernel (equivalent to "bicubic")
"lanczos2"	Lanczos-2 kernel
"lanczos3"	Lanczos-3 kernel

فعالیت ۶: با استفاده از دستورات مناسب، تصویر image1 را ۳۰ درجه و ۴۵ درجه و ۹۰ درجه و ۱۸۰ درجه بچرخانید و در کنار هم ترسیم کنید.

```
image_url = "files/image1.tif";
img1 = imread(image_url);

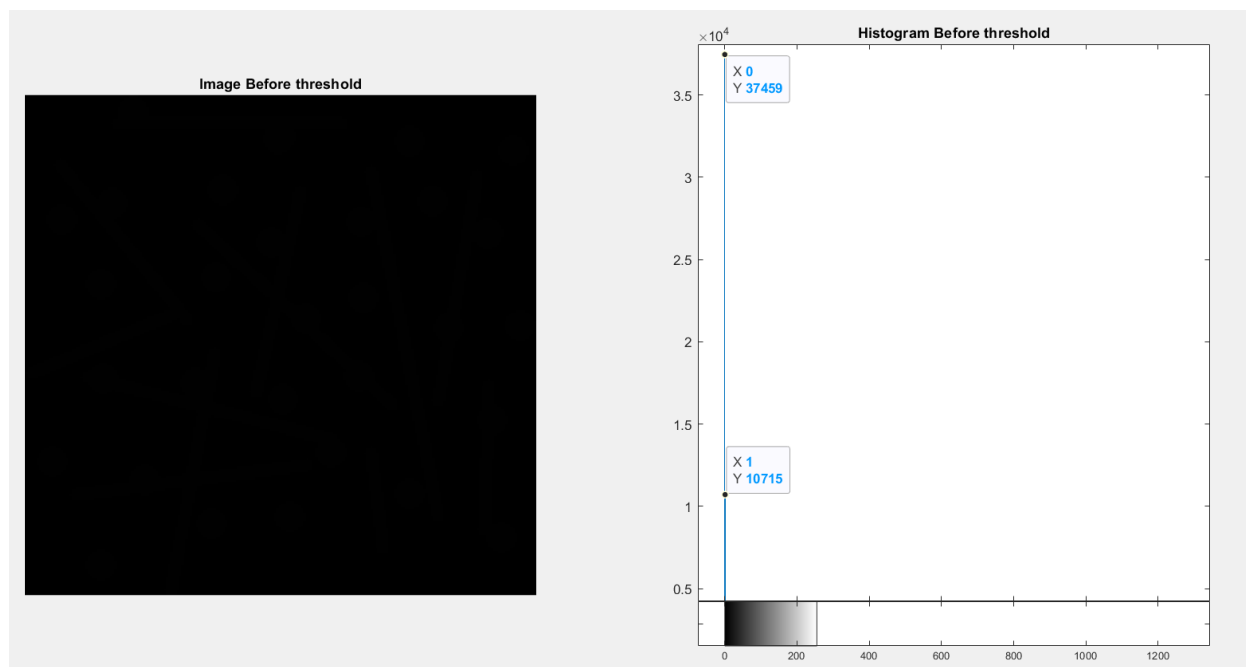
rot_30 = imrotate(img1, 30);
rot_45 = imrotate(img1, 45);
rot_90 = imrotate(img1, 90);
rot_180 = imrotate(img1, 180);

% Display results
figure;
subplot(1,5,1); imshow(img1); title('Original');
subplot(1,5,2); imshow(rot_30); title('Rotate 30°');
subplot(1,5,3); imshow(rot_45); title('Rotate 45°');
subplot(1,5,4); imshow(rot_90); title('Rotate 90°');
subplot(1,5,5); imshow(rot_180); title('Rotate 180°');
```

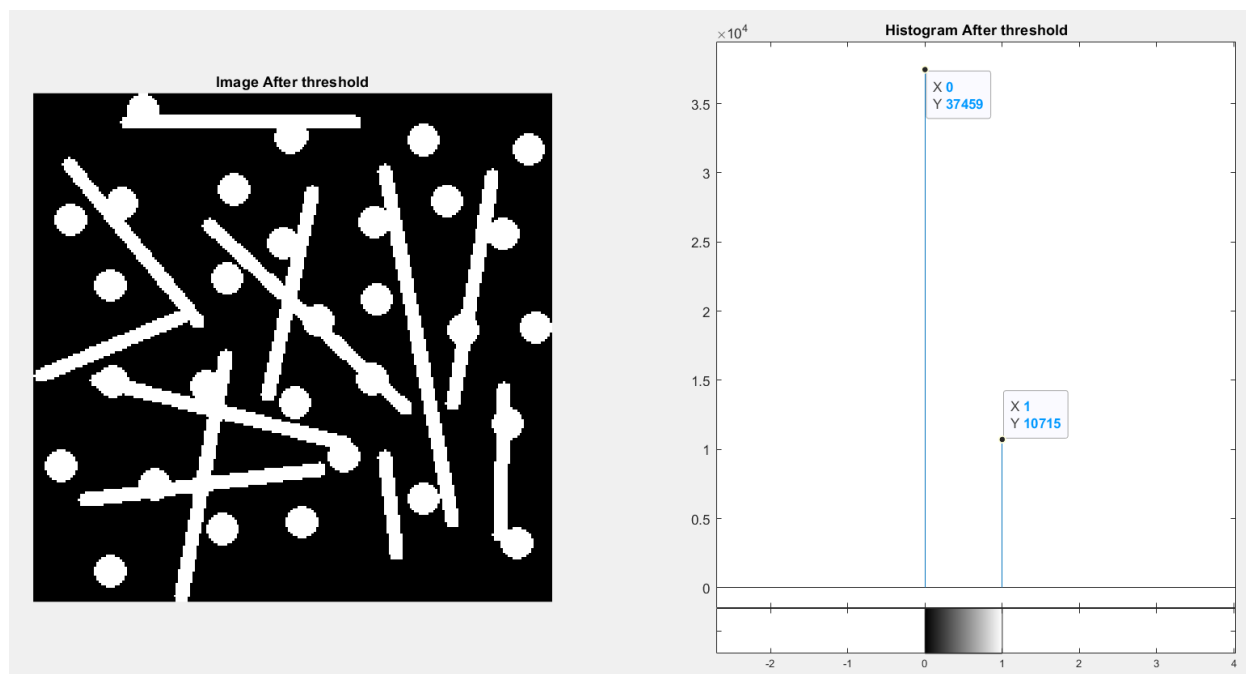


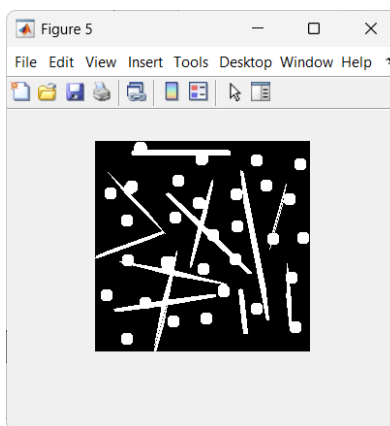
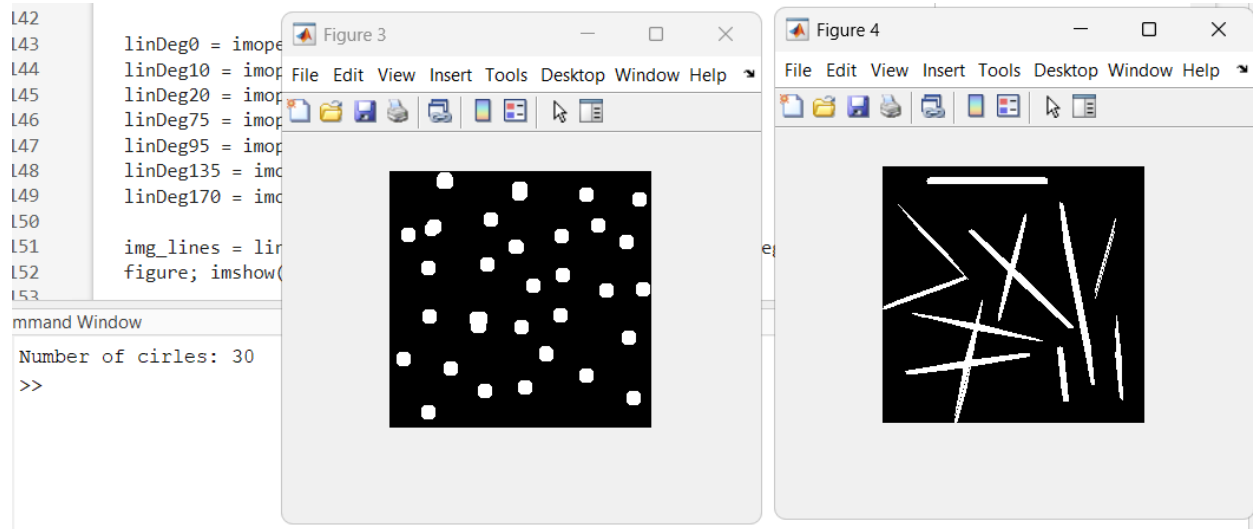
فعالیت ۷: تصویر image4 را خوانده و سپس با استفاده از آستانه‌گذاری بصورت سیاه و سفید تبدیل کنید. سپس با استفاده از دستورات سایش و گسترش مورد نیاز تدايره ها را تفکیک کرده و خطوط و نویزها را حذف کنید. سپس با استفاده از دستورات مناسب connectedcomponents تعداد دایره‌ها را شمارش کنید.

تصویر قبل از آستانه گذاری:



تصویر بعد از آستانه گذاری:





```

image_url = "files/image4.jpg";
img4 = imread(image_url);

figure;
subplot(1,2,1); imshow(img4); title("Image Before threshold");
subplot(1,2,2); imhist(img4); title("Histogram Before threshold");

threshold_val = 0.5;
img_bin = (img4 > threshold_val) * 255;
figure;
subplot(1,2,1); imshow(img_bin); title("Image After threshold");
subplot(1,2,2); imhist(img_bin); title("Histogram After threshold");

img_disks = imopen(img_bin, strel('disk',6));
[~,numCC] = bwlabel(img_disks,8);
disp(['Number of circles: ', num2str(numCC)])
figure; imshow(img_disks);

linDeg0 = imopen(img_bin,strel('line',60,0));
linDeg10 = imopen(img_bin,strel('line',60,10));
linDeg20 = imopen(img_bin,strel('line',60,20));
linDeg75 = imopen(img_bin,strel('line',60,75));
linDeg95 = imopen(img_bin,strel('line',40,95));
linDeg135 = imopen(img_bin,strel('line',60,135));
linDeg170 = imopen(img_bin,strel('line',60,170));

```

```
img_lines = linDeg0 | linDeg10 | linDeg20 | linDeg75 | linDeg95 | linDeg135 |
linDeg170;
figure; imshow(img_lines);

img_final = img_lines | img_disks;
figure; imshow(img_final);
```

بخش ۳: اعمال فیلترهای مختلف بر روی تصاویر

به منظور حذف نویزهای با فرکانس مکانی بالا از فیلتر پایین گذر استفاده می‌کنیم تا تصویر اصطلاحاً نرم‌تر شود. فیلتر کردن تصویر به این صورت است که پنجره‌های با ابعاد معین بر روی تصویر حرکت کرده و روی پیکسل‌ها تاثیر می‌گذارد. ساده‌ترین فیلتر که برای حذف نویز استفاده می‌شود، فیلتر میانگین است. در این نوع فیلتر به جای هر پیکسل در تصویر، میانگین پیکسل‌های همسایگی آن قرار داده می‌شود. بعنوان مثال، عملکرد این فیلتر را برای پنجره‌ی با ابعاد سه در سه در شکل زیر مشاهده می‌کنید.

	a	b	c	
	d	e	f	
	g	h	i	

 $\rightarrow \frac{1}{9}(a + b + c + d + e + f + g + h + i)$

فعالیت ۸: الف) برنامه‌ای بنویسید که برای تصویر image6 فیلتر میانگین با ابعاد 3*3 را اعمال کند. در این تمرین از توابع آماده متلب استفاده نکنید و برنامه را با استفاده از حلقه خودتان بنویسید. ب) سایز فیلتر را تغییر دهید ۵*۵ و ۷*۷ و ۱۱*۱۱ چه نتیجه‌ای می‌گیرید؟



Filtered Image (3x3 Manual)



Filtered Image (5x5 Manual)



Filtered Image (7x7 Manual)



Filtered Image (11x11 Manual)



```
img = imread('files/image6.tif');

if size(img, 3) == 3
    img_gray = rgb2gray(img);
else
    img_gray = img;
end

img_gray = im2double(img_gray);

output_3x3 = manual_average_filter(img_gray, 3);
output_5x5 = manual_average_filter(img_gray, 5);
output_7x7 = manual_average_filter(img_gray, 7);
output_11x11 = manual_average_filter(img_gray, 11);

figure; imshow(img_gray); title('Original Grayscale Image');
figure;
subplot(2,2,1); imshow(output_3x3); title('Filtered Image (3x3 Manual)');
subplot(2,2,2); imshow(output_5x5); title('Filtered Image (5x5 Manual)');
subplot(2,2,3); imshow(output_7x7); title('Filtered Image (7x7 Manual)');
subplot(2,2,4); imshow(output_11x11); title('Filtered Image (11x11 Manual)');

function out_img = manual_average_filter(in_img, k_size)
    [rows, cols] = size(in_img);
    out_img = zeros(rows, cols);

    pad = floor(k_size / 2);

    padded_img = zeros(rows + 2*pad, cols + 2*pad);
    padded_img(pad+1:end-pad, pad+1:end-pad) = in_img;

    weight = 1 / (k_size * k_size);
```

```

for i = 1:rows
    for j = 1:cols
        window = padded_img(i : i + k_size - 1, j : j + k_size - 1);

        pixel_sum = 0;
        for m = 1:k_size
            for n = 1:k_size
                pixel_sum = pixel_sum + window(m, n) * weight;
            end
        end

        out_img(i, j) = pixel_sum;
    end
end
end

```

با افزایش ابعاد فیلتر میانگین مشاهدات زیر حاصل می‌شود:

افزایش شدت تاری: فیلتر میانگین یک فیلتر پایین گذر است که هرچه ابعاد فیلتر بزرگتر میشود تعداد پیکسل های بیشتری در میانگین گیری شرکت میکند و این باعث میشود فرکانس های بالا تضعیف شوند و تصویر مات و تاریک تر میشود.

از بین رفتن جزئیات ریز: در تصویر 3×3 جزئیات کلی تصویر حفظ شده است و در فیلتر 11×11 لبه تیز اشیا محو شده است. کاهش نویز فرکانس بالا: اگر تصویر دارای نویز های دانه ریز باشد فیلتر بزرگتر قدرت بیشتری در محو کردن آنها دارد ولی کیفیت تصویر و وضوح خود تصویر نیز تخریب میشود.

افزایش زمان محاسبات: از نظر ساختاری از آنجا که کد با حلقه هایی تفکیک شده نوشته میشود با بزرگ شدن پنجره فیلتر تعداد تکرار های حلقه ها بیشتر میشود.

فعالیت ۹: به تصویر image7 نویز نمک فلفل با درصدهای مختلف اضافه کنید. سپس با استفاده از تابع fspecial و دستور filter2 انواع فیلترهای مختلف را بر روی آن اعمال کنید و نتیجه را مقایسه کنید.

نوع	توضیحات
average	فیلتر میانگین
disk	فیلتر میانگین دایروی
gaussian	فیلتر گوسی پایین گذر
laplacian	تقریب دوبعدی عملگر لاپلاسی
log	لاپلاس فیلتر گوسی
motion	تقریب حرکت خطی دوربین
prewitt	مشخص کردن لبه
sobel	مشخص کردن لبه

Noisy Image 0.2%



Average Filter



Disk Filter



Gaussian Filter



Laplacian Filter



LoG Filter



Motion Blur



Prewitt Filter



Sobel Filter



Noisy Image 0.5%



Average Filter



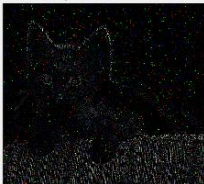
Disk Filter



Gaussian Filter



Laplacian Filter



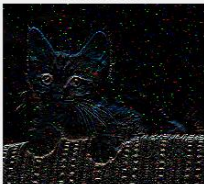
LoG Filter



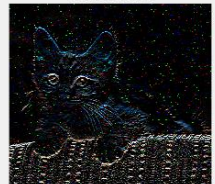
Motion Blur

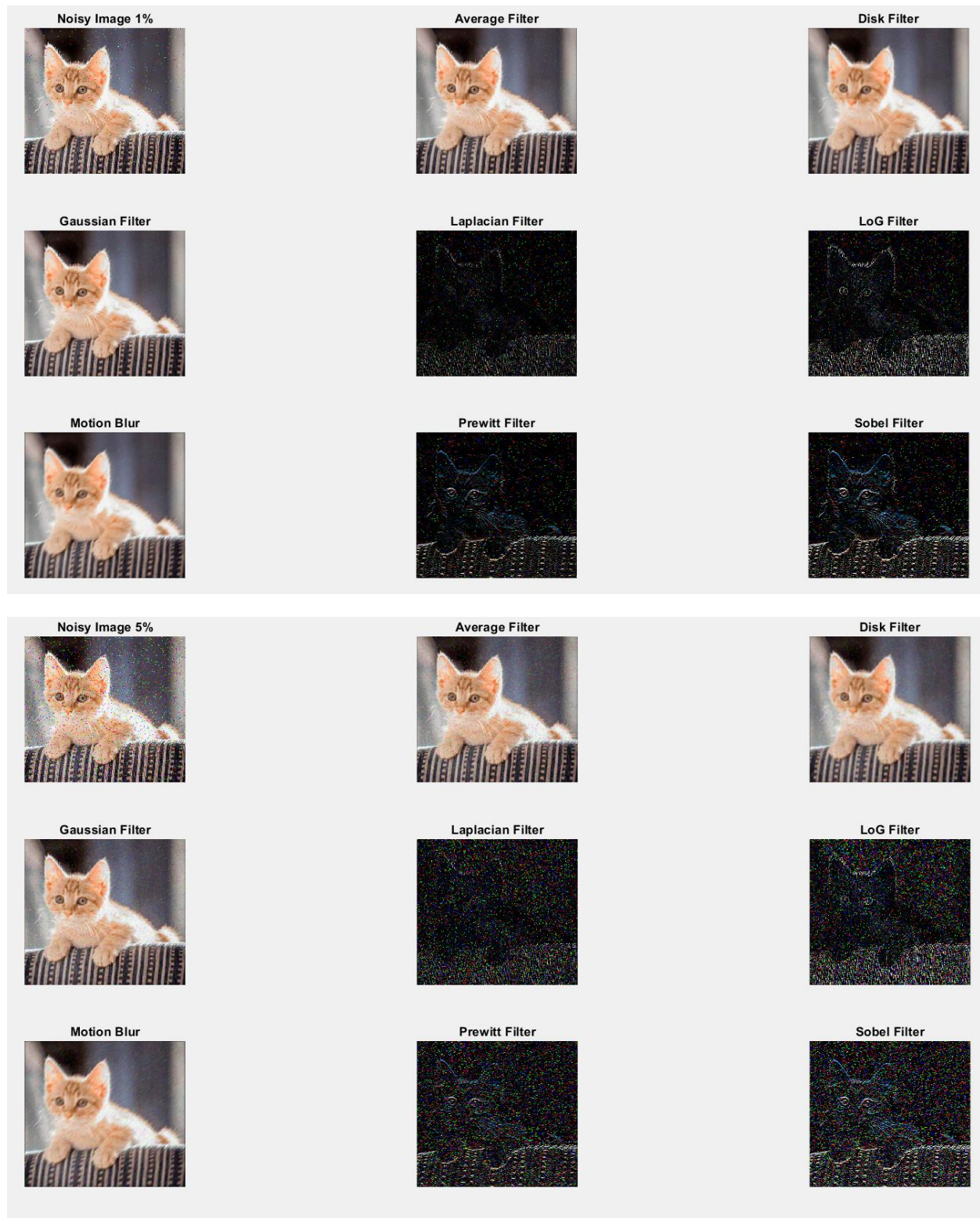


Prewitt Filter



Sobel Filter





```

clc; clear; close all;

[img, map] = imread('files/image7.png');
if ~isempty(map)
    img = ind2rgb(img, map);
end
img = im2double(img);

figure;

```



```

imshow(img);
title('Original Color Image');

noise_density = [0.002 0.005 0.01];

for n = 1:length(noise_density)

    noisy_img = imnoise(img, 'salt & pepper', noise_density(n));

    R_channel = noisy_img(:,:,1);
    G_channel = noisy_img(:,:,2);
    B_channel = noisy_img(:,:,3);

    % Define the filters using fspecial
    h_average = fspecial('average', [3 3]);
    h_disk = fspecial('disk', 3);
    h_gaussian = fspecial('gaussian', [5 5], 1);
    h_laplacian = fspecial('laplacian', 0.2);
    h_log = fspecial('log', [5 5], 0.5);
    h_motion = fspecial('motion', 10, 45);
    h_prewitt = fspecial('prewitt');
    h_sobel = fspecial('sobel');

    f_average = cat(3, filter2(h_average, R_channel, 'same'), ...
        filter2(h_average, G_channel, 'same'), ...
        filter2(h_average, B_channel, 'same'));

    f_disk = cat(3, filter2(h_disk, R_channel, 'same'), ...
        filter2(h_disk, G_channel, 'same'), ...
        filter2(h_disk, B_channel, 'same'));

    f_gaussian = cat(3, filter2(h_gaussian, R_channel, 'same'), ...
        filter2(h_gaussian, G_channel, 'same'), ...
        filter2(h_gaussian, B_channel, 'same'));

    f_laplacian = cat(3, filter2(h_laplacian, R_channel, 'same'), ...
        filter2(h_laplacian, G_channel, 'same'), ...
        filter2(h_laplacian, B_channel, 'same'));

    f_log = cat(3, filter2(h_log, R_channel, 'same'), ...
        filter2(h_log, G_channel, 'same'), ...
        filter2(h_log, B_channel, 'same'));

    f_motion = cat(3, filter2(h_motion, R_channel, 'same'), ...
        filter2(h_motion, G_channel, 'same'), ...
        filter2(h_motion, B_channel, 'same'));

    f_prewitt = cat(3, filter2(h_prewitt, R_channel, 'same'), ...
        filter2(h_prewitt, G_channel, 'same'), ...
        filter2(h_prewitt, B_channel, 'same'));

    f_sobel = cat(3, filter2(h_sobel, R_channel, 'same'), ...
        filter2(h_sobel, G_channel, 'same'), ...
        filter2(h_sobel, B_channel, 'same'));

    figure;
    subplot(3,3,1); imshow(noisy_img); title(['Noisy Image ',
num2str(noise_density(n) * 100) , '%']);
    subplot(3,3,2); imshow(f_average); title('Average Filter');

```

```

subplot(3,3,3); imshow(f_disk); title('Disk Filter');
subplot(3,3,4); imshow(f_gaussian); title('Gaussian Filter');
subplot(3,3,5); imshow(f_laplacian); title('Laplacian Filter');
subplot(3,3,6); imshow(f_log); title('LoG Filter');
subplot(3,3,7); imshow(f_motion); title('Motion Blur');
subplot(3,3,8); imshow(f_prewitt); title('Prewitt Filter');
subplot(3,3,9); imshow(f_sobel); title('Sobel Filter');

```

end

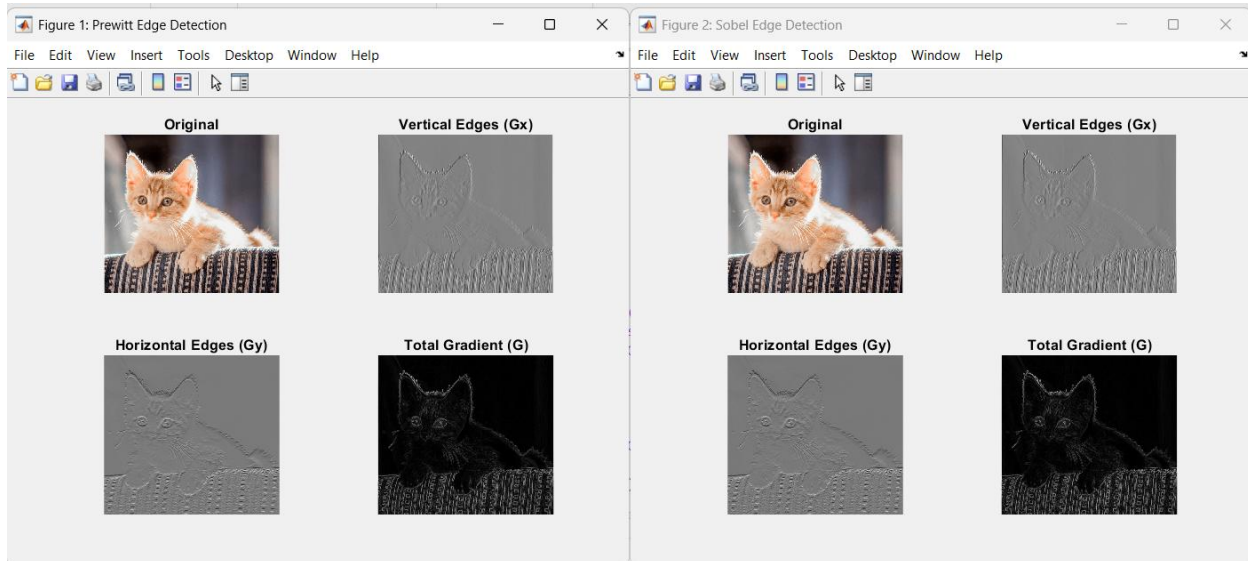
در تصاویر مشخص است فیلترهای نرم کننده و میانگیر گیر شامل **disk, average, gaussian** هستند و باعث پخش نویز در تصویر شده اند و به صورت لکه های تار در تصویر دیده میشوند.

در فیلترهای تشخیص لبه و مشتق گیر این فیلترها شامل لاپلاسین و پریویت و سوبل و **LoG** هستند. این فیلترها به تغییرات ناگهانی در شدت روشنایی پیکسل ها حساس هستند. و این فیلترها روی تصویر نویزی عملکرد بسیار بدی داشتند. به طوری که نویز را به عنوان لبه شناسایی میکردند

فیلتر **Motion** نیز اثر حرکت دوربین در یک زاویه خاص را شبیه سازی میکند و این فیلتر دانه های نویز را به صورت یک خط مورب در می آورد.

فعالیت ۱۰: لبه یابی (فیلتر بالاگذر)، به منظور تعیین لبه ها می توانی با استفاده از روابط مشتق اول و دوم بخش های فرکانس بالای تصویر (یعنی جاهایی که رنگ یا شدن روشنایی تغییر ناگهانی پیدا میکند) را پیدا کنیم. برای این منظور در متلب می توانیم از فیلترهای زیر استفاده کنیم. انواع این فیلترها را بر روی تصویر **image7** اعمال کنید و نتیجه را بررسی کنید.

	1. Prewitt operator	2. Sobel operator
vertical	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$
horizontal	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$



```

image_url = "files/image7.png";

[img7,map] = imread('files/image7.png');
img7 = ind2rgb(img7,map);
img7 = im2double(img7);
img_gray = rgb2gray(img7);

% Convert to double for accurate gradient calculations
img_double = double(img_gray);

% Prewitt masks
Prewitt_vert = [-1 0 1; -1 0 1; -1 0 1];
Prewitt_horiz = [-1 -1 -1; 0 0 0; 1 1 1];

% Sobel masks
Sobel_vert = [-1 0 1; -2 0 2; -1 0 1];
Sobel_horiz = [-1 -2 -1; 0 0 0; 1 2 1];

Gx_prew = conv2(img_double, Prewitt_vert, 'same');
Gy_prew = conv2(img_double, Prewitt_horiz, 'same');
G_prew = sqrt(Gx_prew.^2 + Gy_prew.^2);

Gx_sob = conv2(img_double, Sobel_vert, 'same');
Gy_sob = conv2(img_double, Sobel_horiz, 'same');
G_sob = sqrt(Gx_sob.^2 + Gy_sob.^2);

% Display Prewitt Results
figure('Name', 'Prewitt Edge Detection');
subplot(2,2,1); imshow(img7); title('Original');
subplot(2,2,2); imshow(mat2gray(Gx_prew)); title('Vertical Edges (Gx)');
subplot(2,2,3); imshow(mat2gray(Gy_prew)); title('Horizontal Edges (Gy)');
subplot(2,2,4); imshow(mat2gray(G_prew)); title('Total Gradient (G)');

% Display Sobel Results
figure('Name', 'Sobel Edge Detection');
subplot(2,2,1); imshow(img7); title('Original');
subplot(2,2,2); imshow(mat2gray(Gx_sob)); title('Vertical Edges (Gx)');
subplot(2,2,3); imshow(mat2gray(Gy_sob)); title('Horizontal Edges (Gy)');
subplot(2,2,4); imshow(mat2gray(G_sob)); title('Total Gradient (G)');

```